



Step-by-Step React Router Implementation Guide



Overview

This guide will help you transform a monolithic React component into a well-structured, routed application.

Step 1: Install React Router

In your terminal:

```
bash  
  
npm install react-router-dom
```

What this does: Installs React Router v6, which handles navigation and URL management.

Step 2: Understand the Component Hierarchy

Before (Monolithic):

```
CoursesComponent (1 giant file)  
├── All state management  
├── All data  
├── All UI logic  
└── Conditional rendering for views
```

After (Modular):

```
CoursesApp (Main Router)  
├── CoursesListPage (/)  
├── UnitsPage (/course/:courseId)  
└── ChaptersPage (/course/:courseId/unit/:unitId)  
  
Reusable Components:  
├── CourseCard  
├── UnitCard  
├── ChapterAccordion  
├── LessonItem  
├── CourseMaterial  
└── BackButton
```

Step 3: Import Required Dependencies

At the top of your file:

```
javascript
```

```
import React, { useState } from 'react';  
import { motion } from 'framer-motion';  
import { BrowserRouter, Routes, Route, useNavigate, useParams } from 'react-router-dom';
```

What each import does:

- `BrowserRouter`: Wraps your app and enables routing
 - `Routes`: Container for all your routes
 - `Route`: Defines individual routes
 - `useNavigate`: Hook to programmatically navigate
 - `useParams`: Hook to access URL parameters
-

Step 4: Move Data Outside Components

Extract data to the top level:

```
javascript
```

```
// At the top of your file (outside components)  
const coursesData = [...]; // Your courses array  
const courseMaterials = {...}; // Your materials object
```

Why? Makes data accessible to all components without prop drilling.

Step 5: Create Reusable UI Components

Small, focused components:

CourseCard Component:

```
javascript
```

```
const CourseCard = ({ course, onClick }) => (
  <motion.div onClick={onClick}>
    /* Card UI */
  </motion.div>
);
```

UnitCard Component:

javascript

```
const UnitCard = ({ unit, index, onClick }) => (
  <motion.div onClick={onClick}>
    /* Unit card UI */
  </motion.div>
);
```

Benefits:

- ✓ Reusable across pages
- ✓ Easy to test
- ✓ Clear responsibility

Step 6: Create Page Components

Page 1: CoursesListPage (Home)

javascript

```
const CoursesListPage = () => {
  const navigate = useNavigate(); // Get navigation function

  return (
    <div>
      <h1>My Courses</h1>
      {coursesData.map((course) => (
        <CourseCard
          course={course}
          onClick={() => navigate(`/course/${course.id}`)} // Navigate to units
        </>
      ))}
    </div>
  );
};
```

Key Points:

- Uses `useNavigate()` hook for navigation
 - No props needed - accesses data directly
 - Clean URL: `/`
-

Page 2: UnitsPage (Course Units)

javascript

```
const UnitsPage = () => {
  const navigate = useNavigate();
  const { courseId } = useParams(); // Get courseId from URL

  // Find course by ID from URL
  const course = coursesData.find(c => c.id === parseInt(courseId));

  if (!course) return <div>Course not found</div>;

  return (
    <div>
      <BackButton onClick={() => navigate('/')}/>
      <h1>{course.name}</h1>
      {course.units.map((unit) => (
        <UnitCard
          unit={unit}
          onClick={() => navigate(`/course/${courseId}/unit/${unit.id}`)}
        />
      ))}
    </div>
  );
};
```

Key Points:

- Uses `useParams()` to get `courseId` from URL
 - Finds course data based on URL parameter
 - Navigates to deeper routes
 - URL pattern: `/course/1`
-

Page 3: ChaptersPage (Lessons View)

javascript

```
const ChaptersPage = () => {
  const navigate = useNavigate();
  const { courseId, unitId } = useParams(); // Get both IDs from URL
  const [expandedChapter, setExpandedChapter] = useState(null);

  // Find course and unit
  const course = coursesData.find(c => c.id === parseInt(courseId));
  const unit = course?.units.find(u => u.id === parseInt(unitId));

  if (!course || !unit) return <div>Content not found</div>;

  return (
    <div>
      <BackButton onClick={() => navigate(`/course/${courseId}`)} />
      <h1>{unit.name}</h1>
      { /* Chapters and lessons */ }
    </div>
  );
};
```

Key Points:

- Uses both `courseId` and `unitId` from URL
- Maintains local state for UI (`expandedChapter`)
- URL pattern: `/course/1/unit/2`

Step 7: Set Up the Router

Main App Component:

javascript

```
const CoursesApp = () => {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<CoursesListPage />} />
        <Route path="/course/:courseId" element={<UnitsPage />} />
        <Route path="/course/:courseId/unit/:unitId" element={<ChaptersPage />} />
      </Routes>
    </BrowserRouter>
  );
};

export default CoursesApp;
```

Route Breakdown:

- `/` → Shows all courses
- `/course/:courseId` → Shows units (`:courseId` is a parameter)
- `/course/:courseId/unit/:unitId` → Shows chapters and lessons

Step 8: Understanding URL Parameters

How Parameters Work:

When a user navigates to: `/course/2/unit/3`

javascript

```
const { courseId, unitId } = useParams();
// courseId = "2"
// unitId = "3"
```

Convert to numbers:

javascript

```
const course = coursesData.find(c => c.id === parseInt(courseId));
```

Step 9: Navigation Methods 🌀

Method 1: Programmatic Navigation

```
javascript

const navigate = useNavigate();
navigate('/course/1'); // Navigate to a route
navigate(-1); // Go back one page
```

Method 2: Link Component (Alternative)

```
javascript

import { Link } from 'react-router-dom';

<Link to="/course/1">View Course</Link>
```

Step 10: Testing Your Routes 🧪

Test each route manually:

1. **Home Page:** `http://localhost:3000/`
 - Should show all courses
 2. **Units Page:** `http://localhost:3000/course/1`
 - Should show units for course 1
 3. **Chapters Page:** `http://localhost:3000/course/1/unit/2`
 - Should show chapters for unit 2 of course 1
 4. **Invalid Route:** `http://localhost:3000/course/999`
 - Should show "Course not found"
-

🎯 Complete Flow Example

User Journey:

1. User visits `/` → Sees all courses
2. Clicks "JavaScript Mastery" → Navigates to `/course/1`
3. Sees units for JavaScript course

4. Clicks "Unit 2" → Navigates to `/course/1/unit/2`
5. Sees chapters and lessons for Unit 2
6. Clicks "Back" → Returns to `/course/1`
7. Clicks "Back" again → Returns to `/`

Code Flow:

javascript

// 1. Home page

```
navigate('/course/1')
```

// 2. Units page renders with courseId=1

```
const { courseId } = useParams(); // "1"
```

```
navigate('/course/1/unit/2')
```

// 3. Chapters page renders with courseId=1, unitId=2

```
const { courseId, unitId } = useParams(); // "1", "2"
```

```
navigate('/course/1') // Back to units
```

// 4. Back to units

```
navigate('/') // Back to home
```

Recommended File Structure

```
src/
├── App.js           # Main router
├── data/
│   ├── coursesData.js    # Course data
│   └── courseMaterials.js # Materials data
├── pages/
│   ├── CoursesListPage.js # Home page
│   ├── UnitsPage.js      # Units view
│   └── ChaptersPage.js    # Chapters view
└── components/
    ├── CourseCard.js
    ├── UnitCard.js
    ├── ChapterAccordion.js
    ├── LessonItem.js
    ├── CourseMaterial.js
    └── BackButton.js
```

⚠ Common Pitfalls to Avoid

1. Forgetting to parse URL params:

javascript

// ❌ *Wrong*

```
const course = coursesData.find(c => c.id === courseId);
```

// ✅ *Correct*

```
const course = coursesData.find(c => c.id === parseInt(courseId));
```

2. Not handling missing data:

javascript

// ❌ *Wrong*

```
const unit = course.units.find(...); // course might be undefined
```

// ✅ *Correct*

```
const unit = course?.units.find(...);
```

```
if (!course || !unit) return <div>Not found</div>;
```

3. Wrapping BrowserRouter multiple times:

javascript

// ❌ *Wrong - Only wrap once at the top level*

```
<BrowserRouter>
  <BrowserRouter>
    <Routes>...</Routes>
  </BrowserRouter>
</BrowserRouter>
```

// ✅ *Correct*

```
<BrowserRouter>
  <Routes>...</Routes>
</BrowserRouter>
```

🚀 Next Steps

1. Add 404 Page:

javascript

```
<Route path="*" element={<NotFoundPage />} />
```

2. Add Loading States:

javascript

```
const [loading, setLoading] = useState(true);
```

3. Add Protected Routes:

javascript

```
<Route path="/admin" element={<ProtectedRoute><AdminPage /></ProtectedRoute>} />
```

4. Add Query Parameters:

javascript

```
const [searchParams] = useSearchParams();  
const search = searchParams.get('search');
```

Key Takeaways

✅ **React Router** manages navigation and URLs ✅ **useNavigate** for programmatic navigation ✅
useParams to access URL parameters ✅ **Component separation** improves maintainability ✅ **URL-based routing** enables bookmarking and sharing ✅ **Browser back/forward** works automatically

Useful Resources

- [React Router Documentation](#)
 - [React Router Tutorial](#)
 - [useNavigate Hook](#)
 - [useParams Hook](#)
-

Happy Routing! 🎉